

BIG PICTURE: USER SPACE VS KERNEL SPACE

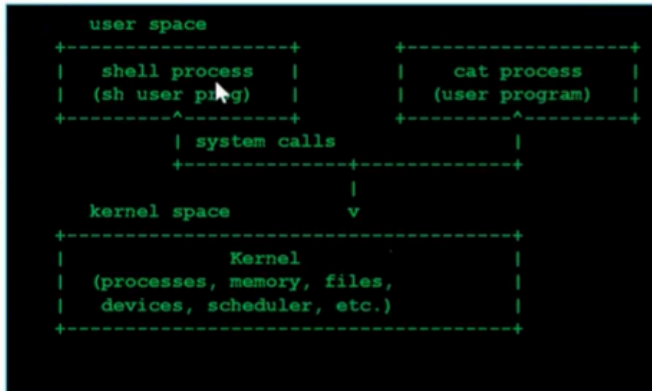


Diagram Reference: [xv6 RISC-V book, Chapter 1 \(Overview and kernel/user split\)](#)

What is xv6?

xv6 is a small teaching operating system used in university operating systems courses. It reimplements an early version of Unix (Version 6) in modern C and RISC-V assembly. The xv6-riscv code is intentionally small and readable, which makes it suitable for students who are seeing OS internals for the first time.

Core ideas:

- **Kernel** runs in supervisor mode and directly controls the CPU, memory, and devices.
- The **user programs** run in user mode with restricted privileges.
- **User programs** invoke **system calls** to request services from the kernel.

Project

A disk image is a single file that pretends to be a whole disk.

Verification

```
yourname@TAHMID: /mnt/c/users/asus/desktop/321Project$ ./mkfs
Created VSFS image 'vsfs.img' (85 blocks).
yourname@TAHMID: /mnt/c/users/asus/desktop/321Project$
```

That terminal output is simply telling you **your `mkfs` program ran successfully and created (or recreated) the filesystem disk image file.**

Here's what each part means:

- **`./mkfs`**
 - You executed the program named **`mkfs`** in the current folder.
 - **`mkfs`** = *make filesystem* (it “formats” a fresh VSFS image file).
- **Created VSFS image '`vsfs.img`' (85 blocks).**
 - The program created a file called **`vsfs.img`** (or overwrote the old one).
 - That file is a **disk image**: a binary file that represents a fake disk.
 - It contains a **VSFS-like filesystem** layout.
 - **85 blocks** means the image is split into 85 fixed-size blocks.
 - In your project spec, **block size = 4096 bytes (4 KB)**.
 - So total image size $\approx 85 \times 4096 = 348,160$ bytes (~340 KB).

What's inside those 85 blocks (based on your PDF layout)

- 1 block superblock
 - 16 blocks journal area (empty initially)
 - 1 inode bitmap block
 - 1 data bitmap block
 - 2 blocks inode table
 - 64 blocks data blocks
- Total = **85 blocks**

Why you run **`./mkfs`** in this project

You run it when you want a **fresh clean vsfs.img** to test your journaling program (**journal.c**) from scratch. If your image got corrupted during debugging, rerun **./mkfs** to reset it.

If you want, tell me what command you ran next (e.g., **./validator**, **./journal create x**) and I'll explain what the output means too.

```
yourname@TAHMID:/mnt/c/users/asus/desktop/321Project$ ./validator
Filesystem 'vsfs.img' is consistent.
yourname@TAHMID:/mnt/c/users/asus/desktop/321Project$
```

That output means your **current vsfs.img** is **structurally OK** according to the provided checker program.

What happened

- You ran: **./validator**
- The validator opened **vsfs.img** and checked the filesystem's metadata (superblock, bitmaps, inode table, root directory, block pointers).

“Filesystem 'vsfs.img' is consistent.” means

The validator did **not** find any of the common corruption problems, like:

- **Inode bitmap mismatch**
(bitmap says an inode is free, but inode table shows it's used — or vice versa)
- **Directory entry problems**
(a dir entry points to an inode that isn't allocated, or invalid inode number)
- **Bad data block pointers**
(inode points to a block outside the data region, or duplicates blocks already used by another inode)
- **Dangling / impossible references**
(e.g., “used” blocks not reachable from any inode)

Why this is important for your journaling project

- Right after running `./mkfs`, you *should* see this message — it confirms the “fresh” image is valid.
- After you run `./journal create <name>` and `./journal install`, you should run `./validator` again:
 - After **create**: still should be consistent (because home metadata shouldn't change yet; only the journal area changes).
 - After **install**: should also be consistent (because journaled metadata updates get applied correctly).

If you show me your output after running `./journal create something`, I can tell you exactly what *should* change (journal header `nbytes_used`) and what must *not* change yet (inode bitmap / inode table / root dir in home locations).

```
yourname@TAHMID:/mnt/c/users/asus/desktop/321Project$ ./journal create test1.txt
Logged creation of 'test1.txt' to journal.
yourname@TAHMID:/mnt/c/users/asus/desktop/321Project$ ./journal create tahmid.txt
Logged creation of 'tahmid.txt' to journal.
yourname@TAHMID:/mnt/c/users/asus/desktop/321Project$
```

You ran your journaling program twice, and each time it **wrote a “create file” transaction into the journal area of `vsfs.img`** — it did **not** actually create the files in the real filesystem metadata yet.

What each line means

`./journal create test1.txt`

- You asked the program to “create a file named `test1.txt`”, but **in journaling mode**.
- Per the spec, `create` should:
 1. read current metadata (inode bitmap, root dir) to decide what would change
 2. compute the updated versions **in memory**
 3. append a **transaction** into the journal:
 - several **DATA records** (each one is a full 4096-byte metadata block image)

- then a **COMMIT** record

4. write **only to the journal blocks** (and journal header)

So it printed:

Logged creation of 'test1.txt' to journal.

meaning: “I recorded the metadata changes needed to create this file, inside the journal.”

`./journal create tahmid.txt`

Same thing again — you appended another transaction into the journal for `tahmid.txt`.

What should be true right now (important)

After these `create` commands:

✓ **Journal should contain 2 committed transactions** (one for each file).

✓ Journal header `nbytes_used` should have increased.

✗ The “real” filesystem metadata (inode bitmap, inode table, root directory block in their home locations) **should still look like before** — because `create` is not allowed to install changes.

What to do next

Run:

- `./validator`
This should still say consistent (because home metadata wasn't changed; journal doesn't usually break consistency checks).

Then run:

- `./journal install`
This will **replay** the committed journal entries into their home blocks (actually creating the files in the real metadata), then clear the journal.

Then run:

- `./validator` again
Should still be consistent.

```
yourname@TAHMID:/mnt/c/users/asus/desktop/321Project$ ./journal install
Installed 2 committed transactions from journal.
yourname@TAHMID:/mnt/c/users/asus/desktop/321Project$ ./validator
Filesystem 'vsfs.img' is consistent.
yourname@TAHMID:/mnt/c/users/asus/desktop/321Project$
```

his output shows your journaling flow is working exactly the way the project spec intends.

1) `./journal install`

Output: `Installed 2 committed transactions from journal.`

Meaning:

- Your journal already contained **2 transactions** (from your earlier commands):
 1. `create test1.txt`
 2. `create tahmid.txt`
- Each transaction was considered **valid** because it had a complete **COMMIT** record.
- `install` then:
 1. **Read the journal sequentially**
 2. For each committed transaction, **replayed every DATA record**
 - i.e., wrote the saved 4096-byte metadata blocks back to their **home block numbers**
 - typical blocks replayed: inode bitmap, inode table block, root directory data block (metadata)
 3. After replaying, it **cleared (checkpointed) the journal** so it becomes empty again (as the spec requires)

So now those two “logged creates” have been **actually applied** to the real filesystem metadata.

2) `./validator`

Output: `Filesystem 'vsfs.img' is consistent.`

Meaning:

- After applying those two transactions, the filesystem's internal metadata is still valid:
 - inode bitmap matches inode table usage
 - root directory entries point to allocated inodes
 - block pointers are valid (no duplicates, no out-of-range blocks)
 - no dangling references

What this implies (nice sanity check)

- ✓ Your **create** is successfully writing transactions into the journal.
- ✓ Your **install** is correctly replaying **only committed** transactions.
- ✓ After replay, the metadata is still correct (validator passes).

Code Explanation

```
1: #include <errno.h>
2: #include <fcntl.h>
3: #include <stdint.h>
4: #include <stdio.h>
5: #include <stdlib.h>
6: #include <string.h>
7: #include <time.h>
8: #include <unistd.h>
```

Header	Primary Purpose	Key Example
<code>errno.h</code>	Error identification	<code>errno</code> , <code>perror()</code>
<code>fcntl.h</code>	File opening flags	<code>O_CREAT</code> , <code>open()</code>
<code>stdint.h</code>	Fixed-size integers	<code>uint32_t</code>
<code>stdio.h</code>	Input/Output	<code>printf()</code> , <code>FILE</code>
<code>stdlib.h</code>	Utility/Memory	<code>malloc()</code> , <code>exit()</code>
<code>string.h</code>	String handling	<code>strlen()</code> , <code>memcpy()</code>
<code>time.h</code>	Clock/Time	<code>time_t</code> , <code>clock()</code>
<code>unistd.h</code>	OS System Calls	<code>fork()</code> , <code>read()</code>

Fcntl ----> file control operations

```
#define FS_MAGIC 0x56534653U
```

```
11: #define JOURNAL_MAGIC 0x4A524E4CU // "JRNL"
```

```
12:
```

```
13: #define BLOCK_SIZE 4096U
```

```
14: #define INODE_SIZE 128U
```

```
15: #define JOURNAL_BLOCK_IDX 1U
```

```
16: #define JOURNAL_BLOCKS 16U
```

```
17: #define INODE_BLOCKS 2U
```

```
18: #define DATA_BLOCKS 64U
```

```
19: #define INODE_BMAP_IDX (JOURNAL_BLOCK_IDX + JOURNAL_BLOCKS)
```

```
20: #define DATA_BMAP_IDX (INODE_BMAP_IDX + 1U)
```

```
21: #define INODE_START_IDX (DATA_BMAP_IDX + 1U)
```

22: `#define DATA_START_IDX (INODE_START_IDX + INODE_BLOCKS)`

23: `#define TOTAL_BLOCKS (DATA_START_IDX + DATA_BLOCKS)`

24: `#define DEFAULT_IMAGE "vsfs.img"`

Line 10: `#define FS_MAGIC 0x56534653U` This is a unique identifier for the File System. The hex value `0x56534653` translates to "VSFS" in ASCII. When the system mounts the disk, it checks for this number to ensure it's reading the correct file system type.

Line 11: `#define JOURNAL_MAGIC 0x4A524E4CU` Similar to above, this identifies the **Journal** section. The hex value `0x4A524E4C` translates to "JRNL". It helps the system verify that the journaling log is valid and not corrupted.

Size Definitions

Line 13: `#define BLOCK_SIZE 4096U` The fundamental unit of storage. The disk is divided into "blocks" of 4096 bytes (4 KB). All reads and writes happen in multiples of this size.

Line 14: `#define INODE_SIZE 128U` The size of a single **inode** (the structure that holds file metadata like size and permissions). Since a block is 4096 bytes, one block can hold $4096 / 128 = 32$ inodes.

Block Counts (Capacity)

Line 15: `#define JOURNAL_BLOCK_IDX 1U` The starting position (index) of the Journal. It starts at Block 1 (Block 0 is usually reserved for the "Superblock").

Line 16: `#define JOURNAL_BLOCKS 16U` The file system allocates 16 blocks specifically for the Journal (to keep track of ongoing changes for crash recovery).

Line 17: `#define INODE_BLOCKS 2U` The total number of blocks reserved for storing inodes. With 2 blocks, this file system can hold a total of 64 files.

Line 18: `#define DATA_BLOCKS 64U` The number of blocks reserved for actual file content (data).

Index Logic (Disk Layout)

These lines calculate **where** each section starts on the disk by adding the previous sections together.

Line 19: `#define INODE_BMAP_IDX (JOURNAL_BLOCK_IDX + JOURNAL_BLOCKS)` The starting index for the **Inode Bitmap** (a map showing which inodes are used/free). It starts immediately after the Journal.

Line 20: `#define DATA_BMAP_IDX (INODE_BMAP_IDX + 1U)` The starting index for the **Data Bitmap**. It starts one block after the Inode Bitmap.

Line 21: `#define INODE_START_IDX (DATA_BMAP_IDX + 1U)` The index where the actual **Inode Table** begins.

Line 22: `#define DATA_START_IDX (INODE_START_IDX + INODE_BLOCKS)` The index where the **Data Region** (actual file contents) begins.

Line 23: `#define TOTAL_BLOCKS (DATA_START_IDX + DATA_BLOCKS)` The total size of the file system image in blocks.

File Defaults

Line 24: `#define DEFAULT_IMAGE "vsfs.img"` The default filename used when creating or mounting the virtual disk file on your computer.

```
26: #define REC_DATA 1
27: #define REC_COMMIT 2
28:
29: #define NAME_LEN 28
30:
31: #define INODE_TYPE_FREE 0
32: #define INODE_TYPE_FILE 1
33: #define INODE_TYPE_DIR 2
```

This block of code defines the **enumerations and limits** used to categorize data and files within the file system. These constants act as "labels" that make the code readable; instead of using raw numbers like `0` or `1`, the programmer uses descriptive names.

Journal Record Types

These constants are used in the **Journaling** system to track the status of a write operation.

- **Line 26: `#define REC_DATA 1`** Indicates a journal entry that contains **data**. When the file system is middle-of-writing, it logs the data here first.
 - **Line 27: `#define REC_COMMIT 2`** Indicates a **commit record**. This is a "final stamp" in the journal. If the system crashes and this record exists, the file system knows the previous data write was completed successfully and can be safely applied to the disk.
-

String Length Limits

- **Line 29: `#define NAME_LEN 28`** This sets the **maximum length for a filename** (28 characters, including the null terminator `\0`).
Why 28? In many simple file systems, this specific number is chosen so that the filename + the inode metadata fits perfectly into a power-of-two size (like the 128-byte `INODE_SIZE` you defined earlier).
-

Inode Types (File Categories)

These defines tell the file system what a specific "inode" (file entry) actually represents.

- **Line 31: `#define INODE_TYPE_FREE 0`** The inode is **empty**. This slot in the Inode Table is available for a new file.
- **Line 32: `#define INODE_TYPE_FILE 1`** The inode represents a **regular file** (like a `.txt` or `.bin` file).
- **Line 33: `#define INODE_TYPE_DIR 2`** The inode represents a **directory** (folder). Directories are essentially special files that contain a list of other filenames and their corresponding inode numbers.

This code defines the **Superblock** structure. In a file system, the Superblock is the "master map." It is usually the very first block on the disk and tells the operating system everything it needs to know about how the disk is organized.

All fields use `uint32_t`, which means they are **unsigned 32-bit integers** (4 bytes each). This ensures the structure is consistent across different computer types.

The Metadata Fields

36: `uint32_t` magic;

Stores the `FS_MAGIC` value (0x56534653). When mounting the disk, the OS reads this first; if it doesn't match, the OS knows the disk is either corrupted or not a VSFS disk.

37: uint32_t block_size;

Stores the size of each block in bytes (defined earlier as 4096). This allows the file system code to know how much data to read at once.

38: uint32_t total_blocks;

The total number of blocks in the entire file system. This defines the "boundary" of the disk so the system doesn't try to read past the end of the file.

39: uint32_t inode_count;

The total number of inodes available in the file system (calculated from INODE_BLOCKS * (BLOCK_SIZE / INODE_SIZE)).

The Layout Map (Pointers)

These fields store the **starting block index** for each section of the disk.

40: uint32_t journal_block;

The block index where the Journal begins.

41: uint32_t inode_bitmap;

The block index where the Inode Bitmap is located.

42: uint32_t data_bitmap;

The block index where the Data Bitmap is located.

43: uint32_t inode_start;

The block index where the actual Inode Table (metadata for files) begins.

44: uint32_t data_start;

The block index where the actual file data begins.

Memory Alignment (Padding)

45: uint8_t _pad[128 - 9 * 4];

This is a "padding" array.

- **The Math:** There are 9 fields above, each 4 bytes ($9 \times 4 = 36$ bytes).
- **The Goal:** The programmer wants the total size of the `struct superbblock` to be exactly **128 bytes** (matching the `INODE_SIZE` or a specific alignment).
- **The Result:** This creates an empty space of 92 bytes. This is useful for future-proofing; if you want to add a new feature later, you can use the padding space without changing the size of the structure or breaking the disk layout.

Summary of the Structure

When the computer "mounts" this file system, it loads this 128-byte structure into memory. It then knows exactly where to jump on the disk to find a file, check for free space, or recover from a crash.

This code defines the **inode** structure. An inode (index node) is the most critical part of a file system because it represents a **file** or **directory**. It stores all the metadata about a file except for its name.

The total size of this structure is carefully padded to be exactly **128 bytes** (matching the `INODE_SIZE` constant you saw earlier).

File Attributes

49: `uint16_t type;`

This identifies what the inode is. It uses the constants defined earlier: 0 for Free, 1 for Regular File, or 2 for Directory. It is a 16-bit unsigned integer (2 bytes).

50: `uint16_t links;`

This tracks the "link count"—how many directory entries point to this specific inode. When this number drops to zero, the file system knows the file is deleted and can free up its space.

51: `uint32_t size;`

The current size of the file in bytes.

Data Pointers

52: `uint32_t direct[8];`

This is an array of 8 "direct pointers." Each element in this array stores the block index of a data block on the disk.

- Since there are 8 pointers and each block is 4096 bytes, this file system supports a maximum file size of $8 \times 4096 = 32,768$ bytes (32 KB).
- If `direct[0]` is 22, the first chunk of the file is located at the 22nd block of the disk.

Timestamps

53: `uint32_t ctime;`

Change Time. Stores the timestamp of when the inode's metadata was last changed (e.g., changing permissions or renaming).

54: `uint32_t mtime;`

Modification Time. Stores the timestamp of when the actual content of the file was last modified.

Padding and Size Calculation

55: `uint8_t _pad[128 - (2 + 2 + 4 + 8 * 4 + 4 + 4)];`

This ensures the struct is exactly 128 bytes. Let's look at the math inside the brackets:

- 2 (type) + 2 (links) + 4 (size) + 32 (8 pointers $\times$ 4 bytes) + 4 (ctime) + 4 (mtime) = **48 bytes used**.
- $128 - 48 = 80$. So, `_pad` is an 80-byte empty array.

Why this structure matters

When you open a file, the OS:

1. Finds the **inode** in the Inode Table.
2. Checks the **type** to see if it's a file.
3. Looks at the **direct[]** array to find exactly where the bytes are stored on the physical disk.

```
58: struct dirent {
59:     uint32_t inode;
60:     char name[NAME_LEN];
61: };
62:
```

This code defines the **Directory Entry** structure. While the **inode** contains the metadata (size, time, location), it does **not** store the filename. In Unix-like file systems, the link between a filename and its data is handled by this **dirent** structure.

Directories in this file system are actually just special files containing a list of these **dirent** structures.

The Fields

59: uint32_t inode;

This is the Inode Number. It acts as a pointer or index into the Inode Table. When you try to access a file, the system looks up this number to find the struct inode we discussed previously. It is a 4-byte unsigned integer.

60: char name[NAME_LEN];

This is a character array (string) that stores the filename.

- It uses the constant **NAME_LEN** (defined earlier as 28).
 - This means every file or folder name can be up to 27 characters long (plus 1 byte for the null terminator **\0**).
-

How it works in practice

Imagine you have a folder named "Documents" containing a file named "notes.txt". The data block for the "Documents" directory would look like a simple table:

Inode Number	Name (string)
5	. (Current directory)
2	.. (Parent directory)
12	notes.txt

When you type **cat notes.txt**, the computer:

1. Scans the directory's **dirent** list for the string "notes.txt".
2. Finds that it corresponds to **Inode 12**.
3. Goes to the **Inode Table** at index 12 to find where the actual data blocks are on the disk.

Total Size

The size of this struct is $4 \text{ bytes (int)} + 28 \text{ bytes (char array)} = 32 \text{ bytes}$.

Because the **BLOCK_SIZE** is 4096 bytes, one single data block assigned to a directory can hold $4096 / 32 = 128$ file entries.

```
63: struct journal_header {
```

nerated 2026-01-06 00:13

ournal.c (full source)

```
64:     uint32_t magic;
65:     uint32_t nbytes_used;
66: };
67:
68: struct rec_header {
69:     uint16_t type;
70:     uint16_t size;
71: };
72:
73: struct data_record {
74:     struct rec_header hdr;
75:     uint32_t block_no;
76:     uint8_t data[BLOCK_SIZE];
77: };
78:
79: struct commit_record {
80:     struct rec_header hdr;
81: };
82:
```

This block of code defines the data structures for the **Journaling System**. Journaling is a "fail-safe" mechanism: before making changes to the actual disk, the file system writes the intended changes here. If the power cuts out, the system can read the journal to finish what it started.

1. The Journal Header

Line 63-66: `struct journal_header` This describes the very beginning of the journal section.

- `uint32_t magic;` Stores `JOURNAL_MAGIC` ("JRNL") to verify this is a valid journal.
 - `uint32_t nbytes_used;` Keeps track of how much of the journal currently contains active, uncommitted data.
-

2. The Record Header (The "Tag")

Line 68-71: `struct rec_header` Every entry in the journal starts with this small 4-byte header. It tells the system what kind of information follows.

- `uint16_t type;` Uses the constants `REC_DATA` (1) or `REC_COMMIT` (2).
 - `uint16_t size;` The total size of the record, which helps the system "skip" to the next record in the list.
-

3. The Data Record (The "Plan")

Line 73-77: `struct data_record` This is the most important part of the journal. It stores a "copy" of a block before it is written to the main disk.

- `struct rec_header hdr;` The tag identifying this as a data record.
 - `uint32_t block_no;` The destination. It tells the system: "This data eventually belongs in Block X on the main disk."
 - `uint8_t data[BLOCK_SIZE];` A full 4096-byte copy of the data to be written.
-

4. The Commit Record (The "Seal")

Line 79-81: `struct commit_record` This is a tiny record that acts as a "Success" flag.

- `struct rec_header hdr;` The tag identifying this as a commit record.

- **The Logic:** When the file system wants to save a file, it writes the `data_record` first, then the `commit_record`.
 - If the system crashes **before** the commit record is written, the file system ignores the data (it's considered "incomplete").
 - If the system crashes **after** the commit record is written, the system knows the data is safe and can be copied to the main disk during recovery.
-

Summary of Journal Layout

When the journal is active, it looks like a sequence of these structures: `[Journal Header]`
-> `[Data Record (Block 5)]` -> `[Data Record (Block 9)]` -> `[Commit Record]`

```

83: static void die(const char *msg) {
84:     perror(msg);
85:     exit(EXIT_FAILURE);
86: }
87:
88: static void read_block(int fd, uint32_t block_no, void *buf) {
89:     if (lseek(fd, block_no * BLOCK_SIZE, SEEK_SET) < 0) {
90:         die("lseek");
91:     }
92:     if (read(fd, buf, BLOCK_SIZE) != BLOCK_SIZE) {
93:         die("read");
94:     }
95: }
96:
97: static void write_block(int fd, uint32_t block_no, const void *buf) {
98:     if (lseek(fd, block_no * BLOCK_SIZE, SEEK_SET) < 0) {
99:         die("lseek");
100:    }
101:    if (write(fd, buf, BLOCK_SIZE) != BLOCK_SIZE) {
102:        die("write");
103:    }
104: }
105:
106: static void read_journal_header(int fd, struct journal_header *jh) {
107:     if (lseek(fd, JOURNAL_BLOCK_IDX * BLOCK_SIZE, SEEK_SET) < 0) {
108:         die("lseek");
109:     }
110:     if (read(fd, jh, sizeof(struct journal_header)) != sizeof(struct journal_header))
111:     {
112:         die("read journal header");
113:     }
114: }
115: static void write_journal_header(int fd, const struct journal_header *jh) {
116:     if (lseek(fd, JOURNAL_BLOCK_IDX * BLOCK_SIZE, SEEK_SET) < 0) {
117:         die("lseek");
118:     }
119:     if (write(fd, jh, sizeof(struct journal_header)) != sizeof(struct journal_header))
120:     {
121:         die("write journal header");
122:     }
123: }

```

Let's go through the provided block of code step by step:

Lines 83-86: **die** function

```

83: static void die(const char *msg) {
84:     perror(msg);
85:     exit(EXIT_FAILURE);
86: }

```

- **Line 83:** This defines a function `die` that takes a single argument `msg` (a string). The `static` keyword means the function can only be used within the file it's defined in.
 - **Line 84:** `perror(msg);` – This function prints the error message to `stderr`. It combines the string `msg` with a description of the most recent error (from `errno`). For example, if `lseek` fails, it will print something like: `Error: lseek: <error_description>`.
 - **Line 85:** `exit(EXIT_FAILURE);` – After printing the error message, this function immediately terminates the program with a failure status (defined by `EXIT_FAILURE` which is typically 1). This halts execution, and no further code is run.
-

Lines 88-94: `read_block` function

```

88: static void read_block(int fd, uint32_t block_no, void *buf) {
89:     if (lseek(fd, block_no * BLOCK_SIZE, SEEK_SET) < 0) {
90:         die("lseek");
91:     }
92:     if (read(fd, buf, BLOCK_SIZE) != BLOCK_SIZE) {
93:         die("read");
94:     }
95: }

```

- **Line 88:** This defines a function `read_block` that reads a block of data from a file descriptor `fd` at a given `block_no` (block number). It writes the data to `buf`, a pointer to the buffer where the data will be stored.
- **Line 89:** `lseek(fd, block_no * BLOCK_SIZE, SEEK_SET)` – This moves the file descriptor `fd` to the position of the specified block number (`block_no`). It seeks to `block_no * BLOCK_SIZE` bytes from the beginning of the file (`SEEK_SET`). If `lseek` fails, it returns a negative value.
 - If `lseek` fails (i.e., it returns a negative value), the program calls the `die` function, printing "lseek" as the error message.

- **Line 92:** `read(fd, buf, BLOCK_SIZE)` – This attempts to read `BLOCK_SIZE` bytes from the file into `buf`. If it doesn't read the correct number of bytes (i.e., it doesn't read `BLOCK_SIZE` bytes), it calls `die` with the message "read".
-

Lines 97-103: `write_block` function

```
97: static void write_block(int fd, uint32_t block_no, const void *buf) {
98:     if (lseek(fd, block_no * BLOCK_SIZE, SEEK_SET) < 0) {
99:         die("lseek");
100:    }
101:    if (write(fd, buf, BLOCK_SIZE) != BLOCK_SIZE) {
102:        die("write");
103:    }
104: }
```

- **Line 97:** This defines the function `write_block`, which writes data to a block at the given block number `block_no` in the file identified by `fd`. The data to be written is in `buf`.
 - **Line 98:** Just like in the `read_block` function, `lseek` moves the file pointer to the correct position for the specified block. If the seek operation fails, it calls `die` with the error message "lseek".
 - **Line 101:** `write(fd, buf, BLOCK_SIZE)` – This writes `BLOCK_SIZE` bytes from the `buf` buffer to the file. If the write operation does not complete the full write (i.e., the number of bytes written is not `BLOCK_SIZE`), it calls `die` with the message "write".
-

Lines 106-113: `read_journal_header` function

```
106: static void read_journal_header(int fd, struct journal_header *jh) {
107:     if (lseek(fd, JOURNAL_BLOCK_IDX * BLOCK_SIZE, SEEK_SET) < 0) {
108:         die("lseek");
```

```

109: }
110: if (read(fd, jh, sizeof(struct journal_header)) != sizeof(struct journal_header)) {
111:     die("read journal header");
112: }
113: }

```

- **Line 106:** This defines the function `read_journal_header`, which reads the journal header into the `jh` structure from a file described by `fd`.
- **Line 107:** The `lseek` function is used to position the file pointer at the start of the journal block, using the index `JOURNAL_BLOCK_IDX` multiplied by `BLOCK_SIZE`. If `lseek` fails, it will call `die` with the message "lseek".
- **Line 110:** `read(fd, jh, sizeof(struct journal_header))` – This reads the size of the journal header (using `sizeof(struct journal_header)`) from the file into the `jh` structure. If it reads an incorrect number of bytes, it calls `die` with the message "read journal header".

Lines 115-121: `write_journal_header` function

```

115: static void write_journal_header(int fd, const struct journal_header *jh) {
116:     if (lseek(fd, JOURNAL_BLOCK_IDX * BLOCK_SIZE, SEEK_SET) < 0) {
117:         die("lseek");
118:     }
119:     if (write(fd, jh, sizeof(struct journal_header)) != sizeof(struct journal_header)) {
120:         die("write journal header");
121:     }
122: }

```

- **Line 115:** This defines the function `write_journal_header`, which writes the journal header from the `jh` structure to the file described by `fd`.
 - **Line 116:** `lseek` is used to move the file pointer to the journal block (again using `JOURNAL_BLOCK_IDX * BLOCK_SIZE`). If it fails, it calls `die` with "lseek".
 - **Line 119:** `write(fd, jh, sizeof(struct journal_header))` – This writes the journal header (from the `jh` structure) to the file. If it doesn't write the full size of the journal header, it calls `die` with the message "write journal header".
-

General Overview:

- **`die` function:** A utility function that handles error printing and program termination.
- **`read_block` and `write_block` functions:** These handle the reading and writing of fixed-size blocks from/to a file.
- **`read_journal_header` and `write_journal_header` functions:** These read and write a journal header from/to a file at a specific location determined by `JOURNAL_BLOCK_IDX`.

In all these functions, `lseek` is used to position the file pointer to the correct block, and `read/write` are used to interact with the file's content. If any operation fails, the program terminates with an error message, making these functions quite robust in error handling.

```
124: static int get_bitmap_bit(const uint8_t *bitmap, uint32_t index) {
```

Generated 2026-01-06 00:13

journal.c (full source)

Page 3

```
125:     return (bitmap[index / 8] & (1U << (index % 8))) != 0;
126: }
127:
128: static void set_bitmap_bit(uint8_t *bitmap, uint32_t index) {
129:     bitmap[index / 8] |= (1U << (index % 8));
130: }
131:
132: static uint32_t find_free_inode(const uint8_t *inode_bitmap, uint32_t max_inodes) {
133:     for (uint32_t i = 0; i < max_inodes; i++) {
134:         if (!get_bitmap_bit(inode_bitmap, i)) {
135:             return i;
136:         }
137:     }
138:     return (uint32_t)-1;
139: }
140:
141: static int journal_append_data(int fd, struct journal_header *jh, uint32_t block_no,
    const void *block_data) {
142:     uint32_t rec_size = sizeof(struct rec_header) + sizeof(uint32_t) + BLOCK_SIZE;
143:
144:     if (jh->nbytes_used + rec_size > JOURNAL_BLOCKS * BLOCK_SIZE) {
145:         fprintf(stderr, "Journal full! Run install first.\n");
146:         return -1;
147:     }
148: }
```

Lines 124-126: `get_bitmap_bit` function

```
124: static int get_bitmap_bit(const uint8_t *bitmap, uint32_t index)
{
125:     return (bitmap[index / 8] & (1U << (index % 8))) != 0;
126: }
```

- **Line 124:** This function, `get_bitmap_bit`, takes a pointer to a bitmap (`uint8_t *bitmap`) and an index (`uint32_t index`) as arguments. A bitmap is essentially a sequence of bits, where each bit may represent a state (e.g., 0 = unused, 1 = used). The

function checks whether a specific bit at the given index is set (i.e., whether it is 1).

- **Line 125:** To access the bit at the `index`, the function calculates which byte in the bitmap the bit belongs to: `index / 8` gives the byte number. The expression `(1U << (index % 8))` shifts the number 1 left by `index % 8` positions to create a mask that isolates the specific bit. It then uses the bitwise AND (&) operator to check if the bit is set (non-zero). If the bit is set, the result is non-zero, and the function returns `true` (1), otherwise it returns `false` (0).
 - **Line 126:** This line marks the end of the function.
-

Lines 128-130: `set_bitmap_bit` function

```
128: static void set_bitmap_bit(uint8_t *bitmap, uint32_t index) {  
129:     bitmap[index / 8] |= (1U << (index % 8));  
130: }
```

- **Line 128:** The function `set_bitmap_bit` sets a specific bit in the bitmap. It takes the bitmap and an index as arguments.
 - **Line 129:** Similar to `get_bitmap_bit`, this line calculates which byte contains the bit (`index / 8`) and then sets the specific bit at the given index by using the bitwise OR (`|=`) operator. It creates a mask by shifting `1U` to the appropriate position (`index % 8`) and ensures that the bit is set (turns it into a 1). If the bit was already set, the operation has no effect.
 - **Line 130:** This line marks the end of the function.
-

Lines 132-138: `find_free_inode` function

```
132: static uint32_t find_free_inode(const uint8_t *inode_bitmap,  
uint32_t max_inodes) {
```

```

133:     for (uint32_t i = 0; i < max_inodes; i++) {
134:         if (!get_bitmap_bit(inode_bitmap, i)) {
135:             return i;
136:         }
137:     }
138:     return (uint32_t)-1;

```

- **Line 132:** This function `find_free_inode` searches for the first free inode in a bitmap. An inode represents a file or directory in a filesystem, and the function looks for the first unused inode (represented by a 0 bit in the bitmap). It takes a pointer to the inode bitmap and the maximum number of inodes as parameters.
- **Line 133:** This begins a `for` loop that iterates over all inodes, from `i = 0` to `max_inodes - 1`.
- **Line 134:** The function calls `get_bitmap_bit(inode_bitmap, i)` to check if the `i`th inode is free. If the bit is 0 (free inode), the function returns `i` as the index of the free inode.
- **Line 135:** If a free inode is found, the function immediately returns its index `i`.
- **Line 138:** If no free inodes are found (i.e., all the bits are set), the function returns `-1` (cast to `uint32_t`), which signifies that no free inode is available.

Lines 141-147: `journal_append_data` function

```

141: static int journal_append_data(int fd, struct journal_header *jh,
uint32_t block_no, const void *block_data) {
142:     uint32_t rec_size = sizeof(struct rec_header) +
sizeof(uint32_t) + BLOCK_SIZE;
143:

```

```
144:     if (jh->nbytes_used + rec_size > JOURNAL_BLOCKS * BLOCK_SIZE)
145:         fprintf(stderr, "Journal full! Run install first.\n");
146:         return -1;
147:     }
```

- **Line 141:** This begins the `journal_append_data` function, which appends data to a journal. It takes a file descriptor (`fd`) for the journal, a pointer to a journal header (`jh`), a block number (`block_no`), and the block data (`block_data`) to append.
- **Line 142:** The size of a record to be appended to the journal is calculated here. It adds up the sizes of the record header (`struct rec_header`), a `uint32_t` (likely for the block number or some other data), and the block data itself (`BLOCK_SIZE`). The total size is stored in `rec_size`.
- **Line 144:** This `if` statement checks if appending a new record would exceed the journal's capacity. It adds the size of the record (`rec_size`) to the current number of bytes used (`jh->nbytes_used`) and compares this value to the total capacity of the journal, which is `JOURNAL_BLOCKS * BLOCK_SIZE`. If the journal would overflow, it prints an error message.
- **Line 145:** If the journal is full, an error message is printed to `stderr` informing the user that the journal is full and they need to install or expand the journal first.
- **Line 146:** The function returns `-1` to indicate that the operation failed because the journal is full.

Summary of the Functions:

1. **`get_bitmap_bit`:** This function checks if a specific bit in the bitmap is set (1) or clear (0).
2. **`set_bitmap_bit`:** This function sets a specific bit in the bitmap to 1.

3. **find_free_inode**: This function searches the inode bitmap for the first free inode (bit set to 0) and returns its index. If no free inode is found, it returns **-1**.
4. **journal_append_data**: This function attempts to append a data block to the journal, checking whether there is enough space. If the journal is full, it prints an error message and returns **-1**.

Corner Cases

Corner Cases & Edge Cases: Metadata Journaling Project

Category 1: Journal Space Management

1.1 Journal Full Scenario

Problem: Journal has limited space (16 blocks = 65,536 bytes)

Test Case:

```
./mkfs
# Each transaction uses ~16,428 bytes (4 DATA records + COMMIT)
# Can fit ~3-4 transactions before journal is full
./journal create f1.txt
./journal create f2.txt
./journal create f3.txt
./journal create f4.txt # Should fail with "Journal full!"
```

Expected Behavior:

- Command should detect full journal
- Print: "Journal full! Run install first."
- Exit without corrupting journal
- NOT append partial transaction

Current Code Check:

```
if (jh->nbytes_used + rec_size > JOURNAL_BLOCKS * BLOCK_SIZE) {  
    fprintf(stderr, "Journal full! Run install first.\n");  
    return -1;  
}
```

✅ This is handled correctly!

1.2 Transaction Spans Multiple Journal Blocks

Problem: A single transaction requires multiple DATA records (4+ blocks)

Test Case:

```
./journal create verylongfilenamexyz.txt
```

```
# One transaction needs: 4 DATA records (4×4104) + 1 COMMIT (4) = 16,420 bytes
```

Expected Behavior:

- Transaction should span across journal block boundaries
- Should be able to read back correctly during install

Potential Bug:

- If journal block boundaries aren't handled properly in `cmd_install()`
- Reading might stop at block boundary

Fix Needed: Your code uses absolute offsets from journal start, so this should work. ✅

1.3 Exactly One Byte Short of Full

Problem: Journal has space for header but not a full transaction

Test Case:

```
# Fill journal to exactly nbytes_used = 65532 (4 bytes short of full)
# Try to create another file
./journal create test.txt # Needs 16,420 bytes, but only 4 available
```

Expected Behavior:

- Should detect insufficient space
- Refuse to create partial transaction

✓ Your boundary check handles this.

Category 2: Inode Allocation Issues

2.1 All Inodes Exhausted

Problem: Filesystem can hold max 64 inodes (inode 0 = root, so 63 files max)

Test Case:

```
./mkfs
for i in {1..64}; do
    ./journal create "file$i.txt"
    ./journal install
done
# 64th file should fail
```

Expected Behavior:

No free inodes

Current Code:

```
if (new_inode_idx == (uint32_t)-1) {
    fprintf(stderr, "No free inodes\n");
    close(fd);
    exit(EXIT_FAILURE);
}
```

✓ Handled correctly!

2.2 Multiple Creates Before Install (Inode Conflict)

Problem: Multiple creates read same on-disk bitmap, allocate same inode

Test Case:

```
./mkfs
./journal create f1.txt
./journal create f2.txt
./journal create f3.txt
./journal install
```

Current Behavior:

- All 3 files get inode 1
- All point to same file (hard links)
- Validator passes but behavior is wrong

Expected Behavior (Ideal):

- Each file should get unique inode
- Requires reading journal to check pending allocations

Status: ⚠ Known limitation in current implementation

2.3 Inode Boundary Cases

Problem: Inode crossing 128-byte boundaries or block boundaries

Test Case:

```
# Create files until inode wraps to second inode block
# First block holds inodes 0-31 (4096/128 = 32 inodes per block)
# Create 33rd file to trigger second inode block
for i in {1..33}; do
    ./journal create "f$i.txt"
done
./journal install
```

Expected Behavior:

- Code should handle inode in different block than root
 - Your code has two cases for this 
-

Category 3: Directory Entry Issues

3.1 Root Directory Full

Problem: Each directory block holds $4096/32 = 128$ entries

Test Case:

```
./mkfs
# Root has "." and ".." already, so 126 files max
for i in {1..127}; do
    ./journal create "file$i.txt"
    ./journal install
done
# 127th should fail
```

Expected Behavior:

Root directory full

Current Code:

```
if (free_slot == -1) {
    fprintf(stderr, "Root directory full\n");
    close(fd);
    exit(EXIT_FAILURE);
}
```

 Handled correctly!

3.2 Duplicate Filenames

Problem: Creating file that already exists

Test Case:

```
./mkfs
./journal create test.txt
./journal install
./journal create test.txt # Should fail
```

Expected Behavior:

File 'test.txt' already exists

✓ Your code checks for this!

3.3 Special Filenames

Empty Filename:

```
./journal create ""
```

Expected: Should reject (directory entry name can't be empty)

Your Code:

```
dirents[free_slot].name[0] = '\0'; // If filename is empty, this happens
```

✗ **Bug:** Empty filename not explicitly checked!

Fix Needed:

```
if (strlen(filename) == 0) {
    fprintf(stderr, "Filename cannot be empty\n");
    exit(EXIT_FAILURE);
}
```

Filename Exactly 28 Characters:

```
./journal create "abcdefghijklmnopqrstuvwxy12" # 28 chars
```

Expected: Should work (NAME_LEN = 28, including null terminator)

Your Code:

```
strncpy(dirents[free_slot].name, filename, NAME_LEN - 1);  
dirents[free_slot].name[NAME_LEN - 1] = '\0';
```

✓ Correctly truncates and null-terminates!

Filename Over 28 Characters:

```
./journal create "verylongfilenamethatexceedstwentyeightcharacters.txt"
```

Expected: Truncate to 27 chars + null terminator

✓ Your strncpy handles this!

Filename with Special Characters:

```
./journal create "file with spaces.txt"  
./journal create "file/with/slashes.txt" # Invalid on many systems  
./journal create "file\0null.txt" # Null byte in middle
```

Current Code: No validation! ⚠

3.4 "." and ".." Filenames

Test Case:

```
./journal create "."  
./journal create ".."  
./journal create "..."
```

Expected Behavior:

- Should reject "." and ".." (reserved)
- "..." might be okay

Current Code: ✗ No check for this!

Fix Needed:

```
if (strcmp(filename, ".") == 0 || strcmp(filename, "..") == 0) {  
    fprintf(stderr, "Invalid filename\n");
```

```
    exit(EXIT_FAILURE);  
}
```

Category 4: Journal Corruption & Recovery

4.1 Uncommitted Transaction (Crash Simulation)

Problem: System crashes after DATA records but before COMMIT

Test Case:

```
./mkfs  
./journal create f1.txt  
# Manually corrupt: remove COMMIT record from journal (hex editor)  
./journal install
```

Expected Behavior:

- Install should discard uncommitted DATA records
- Print: "Warning: X uncommitted DATA records discarded."

 Your code handles this!

4.2 Partial COMMIT Record

Problem: COMMIT record partially written (corruption)

Test Case:

Manually truncate journal to cut COMMIT record in half

Expected Behavior:

- Read fails, transaction discarded

Current Code:

```
if (bytes_read != sizeof(rec_hdr)) {  
    break; // Stops reading  
}
```

✓ Handled!

4.3 Corrupted Journal Magic

Test Case:

```
./mkfs  
./journal create f1.txt  
# Manually change journal magic to 0xDEADBEEF  
./journal install
```

Expected Behavior:

Journal is empty or uninitialized.

✓ Handled!

4.4 Invalid Record Type

Problem: Journal contains unknown record type (e.g., type = 99)

Test Case:

```
# Manually inject record with type = 99
```

Expected Behavior:

- Stop parsing, don't crash

Current Code:

```
} else {  
    // Unknown record type - stop processing  
    break;  
}
```

✓ Handled!

4.5 Record Size Mismatch

Problem: Record claims size = 100 but actually contains 4104 bytes

Test Case:

Corrupt rec_hdr.size field

Expected Behavior:

- Parser gets confused, stops reading

Current Code:

```
offset += rec_hdr.size; // Blindly trusts size field
```

 **Potential Bug:** If size is corrupted, parser jumps to wrong location

Fix Needed: Validate size field:

```
if (rec_hdr.type == REC_DATA) {
    uint32_t expected_size = sizeof(struct rec_header) + 4 + BLOCK_SIZE;
    if (rec_hdr.size != expected_size) {
        fprintf(stderr, "Corrupted DATA record size\n");
        break;
    }
}
```

Category 5: Multiple Transactions

5.1 Multiple Interleaved Transactions

Test Case:

```
./mkfs
./journal create f1.txt # Transaction 1
./journal create f2.txt # Transaction 2
./journal create f3.txt # Transaction 3
./journal install
```

Expected Behavior:

- Install replays all 3 transactions in order
- All 3 files appear

 Your code handles this!

5.2 Empty Journal Install

Test Case:

```
./mkfs  
./journal install # Nothing to install
```

Expected Behavior:

Journal is empty or uninitialized.

 Handled!

5.3 Double Install

Test Case:

```
./mkfs  
./journal create f1.txt  
./journal install  
./journal install # Install again with empty journal
```

Expected Behavior:

- Second install does nothing
- Prints appropriate message

 Handled!

Category 6: File System State

6.1 Missing vsfs.img

Test Case:

```
rm vsfs.img  
./journal create test.txt
```

Expected Behavior:

open: No such file or directory

✓ Handled by `die("open")`

6.2 Corrupted Superblock

Test Case:

```
./mkfs  
# Corrupt superblock magic using hex editor  
./journal create test.txt
```

Expected Behavior:

Invalid filesystem magic

✓ Handled!

6.3 Wrong Block Size in Superblock

Test Case:

```
# Manually change block_size to 2048  
./journal create test.txt
```

Expected Behavior:

- Should detect mismatch and refuse

Current Code:  No check!

Fix Needed:

```
if (sb.block_size != BLOCK_SIZE) {  
    fprintf(stderr, "Invalid block size\n");  
    exit(EXIT_FAILURE);  
}
```

Category 7: Command-Line Handling

7.1 No Arguments

`./journal`

Expected: Usage message 

7.2 Invalid Command

`./journal delete test.txt`

Expected:

Unknown command: delete

 Handled!

7.3 Missing Filename

`./journal create`

Expected:

Usage: `./journal create <filename>`

✓ Handled!

7.4 Extra Arguments

`./journal create f1.txt f2.txt`

Expected: Ignores extra arguments (current behavior)

⚠ **Improvement:** Could warn about extra args

Category 8: Concurrency & Race Conditions

8.1 Multiple Processes

Test Case:

```
./journal create f1.txt &  
./journal create f2.txt &  
wait
```

Expected Problem:

- Both processes read journal header simultaneously
- Both write to journal at same offset
- Journal corruption

Current Code: ✗ No locking mechanism!

Note: Project likely doesn't require concurrency handling, but worth noting.

Category 9: Integer Overflow & Underflow

9.1 nbytes_used Overflow

Problem: If `nbytes_used` wraps around past `uint32_t` max

Test Case: Hard to trigger (would need 4GB journal)

Risk: Low (journal is only 64KB)

9.2 Block Number Out of Range

Problem: DATA record claims block_no = 999 (beyond filesystem)

Test Case:

```
# Manually inject DATA record with block_no = 999
./journal install
```

Expected: Should validate block_no

Current Code: ❌ No validation!

Fix Needed:

```
if (block_no >= TOTAL_BLOCKS) {
    fprintf(stderr, "Invalid block number %u in journal\n", block_no);
    break;
}
```

Summary: Critical Bugs to Fix

Bug	Severity	Fix Needed
Empty filename allowed	Medium	Add strlen check
"." and ".." filenames allowed	Medium	Add reserved name check
No block_no validation in install	High	Validate block_no < TOTAL_BLOCKS
No block_size validation	Low	Check sb.block_size == BLOCK_SIZE
Record size not validated	Medium	Validate rec_hdr.size matches type
Multiple creates share inode	Medium	Check journal for pending allocations

Testing Strategy

Priority 1: Must Test

1. Journal full scenario
2. All inodes exhausted
3. Root directory full
4. Duplicate filename
5. Uncommitted transaction
6. Empty journal install

Priority 2: Should Test

7. Long filenames (>28 chars)
8. Empty filename
9. Special filenames (".", "..", "...")
10. Multiple transactions before install
11. Corrupted journal magic

Priority 3: Nice to Test

12. Invalid block numbers in journal
13. Corrupted record sizes
14. Invalid record types
15. Missing vsfs.img

Project Overview

Sure. In plain, everyday terms: your project is to **add a “write-it-in-a-log-first” safety system** to a tiny fake disk/file-system image, so that **if the system crashes mid-update, the metadata won’t end up half-changed and broken.**

Term Project_ Metadata Journali...

The big idea (layman version)

Think of the file system like a notebook that tracks:

- which “file slots” are taken (inode bitmap),
- details about each file (inode table),
- what names exist in the root folder (root directory block).

Normally, when you “create a file”, the system would directly edit those places. But if power dies halfway, you might get a corrupted state.

So you will implement **metadata journaling**:

1. **First write the intended metadata changes into a journal (a log area).**
2. Only later, when told, **apply (“install”) those logged changes to the real metadata locations.**

Term Project_ Metadata Journali...

This is exactly like:

- writing a “to-do + final confirmation” note in a logbook,
- and only then updating the real ledger.

What you are given

You’re given a pre-built disk image file `vsfs.img` with a fixed layout (blocks are 4KB each). It includes a reserved journal area of **16 blocks** that starts empty.

Term Project_ Metadata Journali...

Also initially, the image has only the **root directory** (inode #0) already set up.

Term Project_ Metadata Journali...

What you must build

You will implement a program called **journal.c** that edits the disk image directly and supports **two commands**:

Term Project_ Metadata Journali...

1) **./journal create <name>**

This command should **NOT actually create the file in the real filesystem metadata yet**.

Instead, it should:

- Look at current metadata to figure out what would need to change to create a new file:
 - find a **free inode** in the inode bitmap,
 - find a **free slot** in the root directory entries,
 - prepare the updated inode info (new inode becomes “file”, etc.).
Term Project_ Metadata Journali...
- Build the **new versions of the affected metadata blocks in memory** (RAM), without writing them to their normal “home” places on disk yet.
Term Project_ Metadata Journali...
- Append **one transaction** into the journal:
 - for each metadata block you changed, append a **DATA record** containing:
 - which “home block number” it belongs to,
 - the **full 4096 bytes** of the updated block,
 - then append a **COMMIT record** at the end to seal the transaction.
Term Project_ Metadata Journali...
- Update the **journal header** (magic + how many bytes are used).
Term Project_ Metadata Journali...

After create:

- The “real” metadata blocks (inode bitmap/inode table/root dir block) are still unchanged.

- Only the journal contains a committed description of what metadata changes should happen later.
Term Project_ Metadata Journali...

Why do it this way?

So if a crash happens right after **create**, the main filesystem isn't half-modified; the journal has a clean record that can be applied later.

2) **./journal install**

This command is the “apply what’s in the logbook to the real ledger” step.

It should:

- Fail if there is no valid journal header (journal not initialized).
Term Project_ Metadata Journali...
- Read the journal records **in order**.
- For each transaction that ends with a valid **COMMIT**:
 - replay every DATA record by writing that block image to its **real home block** in the disk image.
Term Project_ Metadata Journali...
- After applying all committed transactions:
 - **clear the journal** (make it empty again by resetting **nbytes_used** to just the header size).
Term Project_ Metadata Journali...

✓ After **install**:

- The real filesystem metadata is now updated as if the file was created.
 - The journal becomes empty again (checkpointed).
Term Project_ Metadata Journali...
-

What “journal records” look like (in human terms)

Your journal is a big “append-only” log area. It starts with a header:

Journal header

- a magic number (to prove “this is a journal”),
- `nbytes_used` = where the next record should be appended.
Term Project_ Metadata Journali...

Record types

1. **DATA record** = “Here is the complete new version of block X.”
 - includes:
 - record header (type + size),
 - block number (target home location),
 - 4096 bytes of data.
Term Project_ Metadata Journali...
2. **COMMIT record** = “Everything since the last commit is a complete transaction.”
Term Project_ Metadata Journali...

A transaction is only considered valid if its COMMIT record is fully present. (So if a crash happens mid-transaction, install should ignore that incomplete part.)

Term Project_ Metadata Journali...

What counts as “metadata” here

You only journal **metadata blocks**, such as:

- inode bitmap block,

- inode table block(s),
- the root directory data block (because directory entries are metadata).
Term Project_ Metadata Journali...

Also: you must log **whole blocks**, not just small changes. If you add one directory entry, you still log the entire 4096-byte directory block image.

Term Project_ Metadata Journali...

Important constraints you must follow

- The journal is limited (16 blocks). You can append multiple transactions until it fills.
- If there isn't enough space for a full new transaction, `create` must refuse and tell the user to run `install` first.
Term Project_ Metadata Journali...
- Use the provided `validator` program to check the filesystem is consistent:
 - before anything,
 - after `create`,
 - after `install`.
Term Project_ Metadata Journali...

A simple mental example (what “create foo” really does)

When you run:

```
./journal create foo
```

You are basically doing:

1. Find a free inode slot (say inode #1).

2. Mark inode #1 as “used” in inode bitmap (but only in-memory for now).
3. Fill inode #1 info in inode table (type=file, size=0, etc.) (in-memory).
4. Add a directory entry in root dir block: "foo" → inode #1 (in-memory).
5. Write these updated blocks into the journal as DATA records + COMMIT.
6. Do **not** touch the “real” inode bitmap / inode table / root dir in their original places yet.
Term Project_ Metadata Journali...

Later:

`./journal install` copies those journaled block-images into their real locations and clears the journal.

Function Explanation

`read_block` Function

This function reads a specific block from the file descriptor and stores it in a buffer.

- **Line 88:** `static void read_block(int fd, uint32_t block_no, void *buf) {`
 - Defines a static function that takes a file descriptor (`fd`), the block number to read (`block_no`), and a pointer to a buffer (`buf`) where the data will be stored. [↗](#)
 - **Line 89:** `if (lseek(fd, block_no * BLOCK_SIZE, SEEK_SET) < 0) {`
 - Uses `lseek` to move the file offset to the start of the requested block. The offset is calculated by multiplying the block number by the predefined `BLOCK_SIZE` (4096). [↗](#)
 - **Line 90:** `die("lseek");`
 - If `lseek` returns a negative value (indicating an error, such as seeking past the end of the file), the program calls `die` to print the error and exit. [↗](#)
 - **Line 92:** `if (read(fd, buf, BLOCK_SIZE) != BLOCK_SIZE) {`
 - Attempts to read exactly `BLOCK_SIZE` bytes from the file into the buffer. [↗](#)
 - **Line 93:** `die("read");`
 - If the number of bytes actually read does not equal `BLOCK_SIZE`, it indicates a failure or a premature end-of-file, and the program terminates. [↗](#)
-

`write_block` Function

This function takes data from a buffer and writes it to a specific block location on the disk.

- **Line 97:** `static void write_block(int fd, uint32_t block_no, const void *buf) {`
 - Defines a static function for writing. The `buf` parameter is marked `const` because this function should not modify the data it is writing. [↗](#)
- **Line 98:** `if (lseek(fd, block_no * BLOCK_SIZE, SEEK_SET) < 0) {`
 - Similar to the read function, this moves the "write head" of the file descriptor to the exact byte where the block begins. [↗](#)
- **Line 99:** `die("lseek");`
 - Triggers an error exit if the seek operation fails. [↗](#)
- **Line 101:** `if (write(fd, buf, BLOCK_SIZE) != BLOCK_SIZE) {`
 - Attempts to write `BLOCK_SIZE` bytes from the buffer to the file. [↗](#)
- **Line 102:** `die("write");`
 - If the system cannot write the full block (e.g., if the disk is full), it calls `die` to terminate the process safely rather than allowing the filesystem to become corrupted. [↗](#)